

# Business Understanding, Requirements, and Website Design

*Full notes from the conversation, covering business thinking, the requirements workflow, and website psychology and design*

## 1. Are the Best Businessmen the Ones Who Understand Problems, or the Ones Who Solve Them

The idea that the best businessmen and managers are the people who can clearly understand the problem and user requirements, and communicate them well, rather than the people who solve them well, has real truth in it, but it is not fully either-or.

What is being described, understanding a messy situation and turning it into a sharp, correct specification of what needs to be solved, is genuinely the scarcer skill. Most technical people can solve a well-specified problem. Very few people can take an ambiguous situation and frame it correctly. This is real leverage, because a team can spend months building an elegant solution to the wrong problem.

Where a pure either-or framing breaks down is that the best businesspeople and managers usually are not purely translators who stay away from solving. They tend to have enough hands-on fluency to:

- Sanity-check whether a proposed solution actually addresses the problem they framed
- Know which parts of the problem are hard versus easy, so they do not set impossible expectations
- Earn credibility with the people actually doing the solving, because they have felt the difficulty themselves

A manager who can articulate the problem beautifully but has no feel for solution difficulty tends to make bad tradeoffs, such as impossible deadlines, underestimating technical debt, or picking flashy but wrong solutions because they sound right.

A sharper version of the original claim is that understanding and communicating is the higher-leverage skill, and it is undervalued relative to solving, but the best operators usually have just enough solving experience to keep their understanding honest. Pure translators without any solving instinct tend to drift into misreading what is actually hard.

## Should the Businessman Be Able to Solve the Problem Himself

Not necessarily solve it themselves, but they should understand how it gets solved well enough to judge quality, spot bad work, and negotiate tradeoffs.

There is a difference between being able to execute the solution and being able to evaluate the solution. A business person does not need to be able to write the code, design the circuit, or run the machine learning pipeline. But if they cannot tell a good solution from a mediocre one, they are at the mercy of whoever is building it. They will approve overengineered work, get talked into bad timelines, or reject something good because it does not look like what they expected.

A few reference points on this:

- Steve Jobs could not write software, but he had enough taste and technical literacy to reject bad UI decisions and push engineers past good enough
- Many failed startups have a founder who was purely vision and communication, and got completely steered by whoever they hired, because they had no way to independently judge whether the technical work was actually good
- Conversely, some of the worst managers are ex-solvers who never let go. They keep re-solving problems themselves instead of enabling others, which does not scale

This can be framed as a spectrum rather than a binary:

- Full solving ability: not required, and often wastes their time and attention as the company grows
- Deep enough fluency to evaluate solutions and ask sharp questions: this is the real requirement
- Zero technical grounding, pure vision and communication: risky, because they cannot tell good work from bad

Where on that spectrum someone needs to sit depends on the domain and company stage. Early-stage technical startups often need a founder closer to level one, because there is no one else to check the work. Larger organizations can afford leaders who are purely at level two, because they have technical leads underneath them they trust.

## 2. Workflow for Understanding a Problem, Gathering Requirements, and Building an SRS

This workflow starts from gathering complaints and ends at a full SRS document.

### Step 1: Gather Complaints

- Collect from multiple channels, do not rely on one source: direct interviews, support tickets or logs, observation of actual use, since watching someone work is more honest than asking them, and surveys for scale
- Capture verbatim, in the person's own words, do not paraphrase or clean it up yet, because paraphrasing early is where an assumption about the fix gets accidentally injected
- Log metadata with each complaint: who said it, their role or group, how often it happens to them, how severe they say it is
- Deliberately over-collect at this stage. It is easier to discard noise later than to go back and ask what else bothers you a second time
- Do not respond to or debate complaints while collecting. Just log them. Arguing at this stage taints the next person's honesty

### Step 2: Cluster and De-duplicate

- Group similar complaints together across different people, since five people describing the same friction in different words is one issue, not five
- Note frequency: how many distinct people raised this, not just how many times it was mentioned
- Note intensity: is this an annoyance or a blocker
- Flag complaints that only one person raised, not to discard them, but to look at them differently. It could be an edge case that matters, or a personal quirk

### Step 3: Classify as Alloplastic, Autoplastic, or Mixed

This classification is about whether the system should change, alloplastic, or the person's expectations and habits should change, autoplastic.

- Alloplastic candidates: the environment is genuinely the problem. Signs include multiple unrelated users hitting the same friction, a costly or error-prone workaround, or a complaint pointing at something objectively broken or missing
- Autoplastic candidates: the person's mental model, habit, or expectation is the mismatch, not the system. Signs include a single user's idiosyncratic workflow, unfamiliarity with a better existing path, or resistance to a reasonable new process
- Ambiguous or contested: the hardest and most important bucket. Often both are partially true, the system has a rough edge and the user's expectation is off. This is where most real disagreement lives

For the ambiguous bucket, do not just pick a side. Weigh:

- Cost to change the system versus cost to retrain or communicate to the user
- How many people are affected by leaving it autoplastic, meaning how many people have to adapt
- Whether the autoplastic fix is even fair. Sometimes what looks like a training problem is really the system being badly designed, and asking users to adapt is just avoiding the harder alloplastic fix
- Precedent. If this is made alloplastic, does that commit the team to fixing every future variant of this complaint too

The output of this step is each complaint tagged alloplastic, autoplastic, or mixed, with a one-line justification. Mixed ones get a stated tradeoff decision, not a shrug.

## Step 4: Route the Output

- Alloplastic and mixed-resolved-toward-system items proceed to requirements conversion
- Autoplastic and mixed-resolved-toward-user items go into a separate track: training material, documentation, onboarding flow, change management. This is real work, just not SRS work

## Step 5: Convert to Requirements, Plasticity Conversion

- Rewrite each surviving complaint as one atomic, testable requirement, one sentence per requirement
- Classify each as functional, what the system does, or non-functional, performance, security, usability constraints
- Attach an acceptance criterion to each one: how would you test that this requirement is met
- Prioritize using MoSCoW: Must have, Should have, Could have, Won't have. This is where conflicting requirements get resolved, usually by trading off against priority
- Remove ambiguous words like fast, user-friendly, robust, and replace with numbers or specific conditions

## Step 6: SRS Design

Structure the document, a standard IEEE-830-ish shape works fine:

- Introduction: purpose, scope, definitions
- Overall description: product perspective, user classes, constraints, assumptions
- Specific requirements: functional, non-functional, and external interface requirements
- Appendices: diagrams, use case, class, sequence
- Trace each requirement back to the stakeholder who raised it. A simple traceability table helps a lot later when scope disputes happen
- Get sign-off from at least one representative of each stakeholder group before moving to design

## How to Gather Facts for Each Part of the SRS

Introduction:

- Purpose: pulled from the problem statement in step one. Restate it formally, what the system is for, in one paragraph
- Scope: pulled from the boundary set in step one, what this will not touch. Cross-check against the routing in step four, anything routed to the autoplastic or training track should be explicitly listed as out of scope
- Definitions and acronyms: built as you go. Every time a stakeholder used a domain-specific term during interviews, log it with their definition, not an assumed one. Terms often mean slightly different things to different stakeholder groups, and that mismatch itself is worth a note
- Intended audience and references: administrative, list who will read the document and any relevant standards or regulations

#### Overall Description:

- Product perspective: does this replace something, extend something, or is it brand new. This comes from the problem's root cause and any current process notes gathered during complaint collection
- User classes and characteristics: comes directly from identifying involved people. Each group mapped by influence and interest becomes a user class here. Note technical skill level and frequency of use, since this affects non-functional requirements like UI complexity
- Constraints: two sources, anything a stakeholder mentioned as a hard limit during interviews such as budget, deadline, or integration needs, and anything identified in classification as cost to change is too high, a constraint being accepted rather than solved
- Assumptions and dependencies: anything taken at face value without verifying. Be honest here. If a stakeholder's complaint was assumed to apply to everyone in their group but only two of ten were interviewed, that should be written down as an assumption, not a fact

#### Specific Requirements:

- Functional requirements: one per surviving alloplastic or mixed complaint from step five. Write as the system shall do this action. Trace it back to the complaint cluster ID from step two so the reasoning is auditable later
- Non-functional requirements: usually do not come from single complaints directly, but emerge across the cluster. If five complaints all resolve to this is slow, that is one non-functional requirement, response time under a stated number of seconds, rather than five functional ones
- External interface requirements: gathered by asking each user class what they already use, existing software, hardware, other systems they need this to talk to. This is a targeted follow-up interview, not something that comes from general complaint gathering

#### Traceability Table:

- Built as a byproduct, not a separate research effort. Just a table with columns, requirement ID, complaint cluster ID, stakeholder who raised it, priority. If clean IDs are kept at every step, this table assembles itself instead of requiring the origin to be re-derived

#### Diagrams:

- Use case diagrams: derived from the functional requirements list. Each system shall statement usually maps to one or more use cases, actors pulled from user classes
- Class diagrams: derived from nouns that repeat across the requirements, which tend to become entities
- Sequence diagrams: pick the three to five highest-priority, must-have, use cases and walk through them step by step, usually by asking the relevant stakeholder to narrate their ideal flow

#### Sign-off:

- Not really a fact-gathering step. Go back to the specific person or people who raised each must-have complaint and confirm the requirement as written actually addresses what they meant. This catches translation errors before they become expensive

### 3. What a Website Actually Is: Psychological and Media Aspects

A website does not map cleanly onto any older medium, and this mismatch is exactly why web design was such a mess for its first fifteen to twenty years. People kept forcing it into inherited metaphors that did not fit.

Going through the common candidates, each is partially right and partially wrong, and the gap between them is where the actual medium lives.

- Poster: captures the fact that a website has a visual hierarchy, a first impression, a hero moment. But a poster is static and single-purpose, one message, one glance, done. A website is entered, not glanced at, and invites navigation a poster never does

- Book: captures depth, sequence, chapters as pages. But a book has a fixed linear order the author controls. A website's structure is a graph, not a line. The user picks their own path, and a huge amount of design work goes into making that graph legible through navigation, breadcrumbs, and search, rather than forcing a sequence
- Newspaper: closer than the first two. Captures the idea of many independent stories coexisting on one surface, browsable rather than linear, with editorial hierarchy such as what sits above the fold. But a newspaper is broadcast, same content to everyone, published once, then dead. A website is interactive and often personalized or live
- Store: captures the transactional, goal-directed aspect, you go in wanting something specific and leave. But most websites are not stores. Even stores are also newspapers, posters, and conversations at once

What a website actually is: a space you visit, not an object you consume. That is the real category shift. Every prior medium, poster, book, newspaper, is something a person receives, it comes to them, fixed. A website is closer to a building or a room: you enter it, you move through it, its layout implies behavior such as where the door is or where the checkout counter is, and it can react to you in real time.

## The Psychological Angle

This follows from the space framing:

- Wayfinding, not reading, is the dominant mental mode. Users behave like people in an unfamiliar building, scanning for exits and landmarks such as the nav bar, logo, and search before committing to read anything. This is why F-pattern eye-tracking studies show people scan rather than read on first contact
- Agency and control matter in a way they do not for a book. A reader trusts the author to lead them, a website user expects to lead themselves, and gets anxious or distrustful the moment control is taken away, such as autoplay video, a forced signup wall, or no visible back button
- Trust is spatial before it is verbal. People judge a website's credibility from layout and visual polish in a fraction of a second, before reading a word, much closer to how people judge a physical storefront than how they judge a book's prose
- Liveness creates a different relationship to time. A book is finished, and the reader knows it. A website is presumed to be current unless it visibly proves otherwise, such as broken links or old dates, so staleness reads as abandonment or untrustworthiness in a way a ten-year-old novel never does

So the direct answer is that a website is closer to architecture with pages inside it than to any print medium. The print metaphors, poster, book, newspaper, describe what is inside the rooms. The space you navigate idea describes the building itself, and that is the part that actually determines whether people trust it, get lost in it, or leave.

## 4. Why a Business Should Use a Website Beyond General Media

This reasoning falls directly out of the website as space idea from the previous section.

### Theoretical Framing: Owned Versus Rented Attention

The clearest theoretical distinction is owned media versus rented media.

- Social media, marketplaces, review platforms, and ad networks are all rented land. A business builds a following, an audience, a storefront, but the platform owns the rules, the algorithm, the layout, the access to the audience, even whether the business gets to keep that audience tomorrow. Platforms change algorithms, ban accounts, throttle reach, or shut down entirely, and the business has zero recourse
- A website is owned property. Nobody can deprioritize it in a feed, demonetize it, or change the rules of what is shown next to the brand. This maps onto institutional economics' idea of asset specificity and control, owning the asset removes a dependency risk that renting always carries

This alone is a strong argument independent of psychology, a business that only exists on rented platforms has no fallback if the platform changes terms, and no single, stable place to send someone who wants to actually evaluate them.

## Psychological Angle: What Having a Place Does to a Visitor

Going back to the website as space idea, a business having a website versus only social presence changes how a visitor's mind processes them, in a few specific ways:

- **Costly signaling:** in signaling theory, an investment that is expensive to fake is trusted more than one that is cheap to fake. A social media account is nearly free to create, so it signals almost nothing about legitimacy on its own. A well-built website implies a business spent real time and money establishing itself, which unconsciously reads as this is a real, serious operation, the same way a physical storefront with a lease signals more permanence than a folding table at a market
- **Autonomy and self-determination:** self-determination theory holds that people engage more deeply and trust more when they feel a sense of autonomy rather than being led. On social media, the visitor is inside the platform's flow, swiping, algorithmically fed, never fully arriving anywhere. A website hands the visitor control, they choose what to click, what to read, when to leave. That felt agency increases trust and consideration, versus the passive, drive-by nature of a feed impression
- **Context integrity:** on a platform, a business's content sits next to competitors' ads, unrelated content, comments, and platform-inserted material the business does not control. This is sometimes called context collapse, the business's message is filtered through an environment it did not design, and the visitor's impression is shaped by all of that surrounding noise. A website is the one place where the business controls all of the context, no competitor ad next to their pricing page, no hostile comment under their claim
- **Persistence versus ephemerality:** social content is built for a feed's timescale, hours or days of relevance before it is buried. A website is a stable reference point that exists regardless of when someone arrives. Psychologically, this matters because trust-building often requires repeated, low-effort access to consistent information, a person might see a social post once, forget it, then later search the business by name and expect to find something durable. If nothing persistent exists, that follow-through moment fails
- **Legitimacy heuristics:** people use simple mental shortcuts to judge credibility fast, this is well documented in web-credibility research, such as Stanford's studies on how users judge site trustworthiness in seconds. A domain name, a coherent design, an about page, a real address, are all legitimacy cues that social profiles either cannot provide or provide much more weakly, since anyone can spin up a profile in minutes

## Where the Two Actually Differ in Function

Theoretically, it helps to think of it as two different jobs rather than one being better:

- Social and general media are optimized for discovery and reach, getting in front of people who do not yet know you exist, in an environment they are already spending time in
- A website is optimized for conversion and trust-building, the place discovery eventually routes to, where a person actually decides whether to believe, buy, or commit

A business relying only on social media has strong discovery but a structurally weak trust and conversion layer, because it is asking people to make a decision inside a rented, noisy, ephemeral space. A business with only a website has the opposite problem, strong trust infrastructure, but no discovery engine driving anyone there. This is why in practice they are used together, not as substitutes, social gets attention, the website converts it into something durable.

## 5. Solving the Blank Page Problem When Designing a Website

The blank page problem, where only a few core functional elements come to mind and the pages end up feeling empty, is actually a diagnostic. **It means the design is coming straight from functional requirements only, skipping the trust and context layer entirely.** Functional-minimum thinking gives a login form and a submit button, but nothing that answers the psychological questions a visitor is silently asking, such as who are you, can I trust this, am I in the right place. This is usually the missing content, not more features. The fix uses the same workflow structure as before.



## Apply Step 1, Problem, per Page

For each page, write one sentence: this page exists so a visitor can decide or do a stated thing. A functional-minimum page answers only the do part. Every page also needs to resolve the decide part first, trust, orientation, relevance, before the visitor is willing to act. That is usually the missing content.

## Apply Step 2, Identify People, per Page

The earlier stakeholder list, users, stakeholders, affected parties, still applies, but now ask which of these people land on this specific page, and what they need before they will trust it enough to act. A first-time visitor to a homepage needs different reassurance than a returning customer on a checkout page. This alone generates content, testimonials for the skeptical first-timer, order summary clarity for the returning buyer.

## Apply Step 3, Gather Complaints, Proactively About Absence

Earlier the workflow gathered complaints about what is broken. Here, gather complaints about what is missing, run this as a real exercise, not guesswork:

- Show the bare-minimum page to three to five people and ask what they would want to know before they would trust or use it
- Their answers are the content gaps. This is literally step one of the earlier workflow, gather complaints, just aimed at content instead of features

## Apply Step 3's Classification, Repurposed for Content

Ask of each blank space, is this blank because content does not exist yet, alloplastic, meaning it needs to be created, or because the visitor is assumed not to need it, autoplatic. Most blank pages are the second case. The visitor is being asked to adapt to sparse information instead of the site providing what builds trust. That is usually the wrong tradeoff for anything except the most functional utility tools.

## Content Categories to Fill the Gap

Grouped by what psychological job they do, based on the trust and legitimacy points from earlier:

Legitimacy signals, answers is this real:

- About or team info, physical address or location context, years in business, credentials or certifications, press mentions or partner logos

Social proof, answers do other people trust this:

- Testimonials, reviews, case studies, usage numbers, client logos

Orientation content, answers am I in the right place, what do I do here:

- A clear one-line value statement above the fold, visual hierarchy showing primary action versus secondary info, breadcrumbs or navigation that show where the visitor is

Risk reduction, answers what happens if this goes wrong:

- FAQ, return or refund policy, guarantee, contact info, support availability

Depth for the undecided, answers I am not ready to act yet, but I want to understand more:

- How it works, comparison content, detailed feature explanation, blog or resources

Not every page needs every category, that is where the per-page audience of visitors filters it down. A checkout page needs risk reduction and almost nothing else. A homepage needs legitimacy, social proof, and orientation.

## Apply Step 6, SRS Equivalent, a Content Requirements Table

Just like before, build a table: page, visitor type, question they need answered, content element, priority, Must, Should, or Could. This turns the page feels blank into a traceable gap. It becomes possible to see exactly which visitor question has no content answering it, instead of vaguely feeling like something is missing.

### Core Mistake to Avoid

Treating functional and content as separate design phases. Functional-minimum thinking optimizes for the visitor who already decided to act. Most visitors have not decided yet, the content is what gets them there.